

The AI Ops Security Playbook

5 Strategies to Secure Your Operations
Without Slowing Down Innovation

Riccardo Bozzato

Head of Operations & AI Security Builder

riccardobozzato@gmail.com • [linkedin.com/in/riccardobozzato](https://www.linkedin.com/in/riccardobozzato)

Introduction

After years running operations at a fintech-adjacent company while building security tools in my spare time, I've learned what actually works. This playbook distills the tactics I use daily — no theory, just field-tested practices.

My name is Riccardo Bozzato. I am a Head of Operations with a passion for building secure, resilient systems. My work spans cloud infrastructure, incident response, compliance automation, and open-source security tooling. I believe that security and operations are not opposing forces — when aligned correctly, they amplify each other.

This playbook is written for ops leaders, platform engineers, and security builders who need practical answers. Every strategy here has been tested in production. Use what fits, adapt what doesn't, and keep shipping.

~~Strategy 1: The Ops-Security Alignment Framework~~

Most security initiatives fail because ops sees them as blockers. When a new control slows down a deploy or adds friction to incident response, teams route around it. The result? Shadow IT, alert fatigue, and a false sense of safety.

The Ops-Security Alignment Framework solves this by requiring every security control to map directly to an ops metric. If it doesn't improve uptime, reduce response time, or lower error rate, it needs rethinking — or it gets cut.

How it works:

- Map each security control to an ops KPI (uptime, MTTR, error budget)
- Replace manual approval gates with automated policy checks
- Measure security adoption by ops velocity, not compliance scorecards
- Review quarterly: if a control doesn't move an ops needle, deprecate it

Key takeaway: Security shouldn't be a gate. It should be a gear in the machine.

~~Strategy 2: 3 Automation Plays That Save 10+ Hours/Week~~

The biggest time sink in ops-security is manual toil. These three plays eliminate the busywork so you can focus on what matters.

Play 1: Automated Incident Triage

Trigger: Alert fires in monitoring system (e.g., Grafana, Prometheus).

Automation: Logs are fed to an AI classification layer that enriches the alert with severity, affected service, and probable cause. A Slack notification is posted with a structured summary and a link to the runbook.

Time saved: ~4 hours/week per on-call engineer.

Play 2: Compliance as Code

Trigger: A pull request modifies infrastructure definitions (Terraform, Kubernetes manifests).

Automation: Policy-as-code tools (e.g., Open Policy Agent, Checkov) scan the changes against compliance baselines (SOC2, ISO 27001, internal controls). The pipeline blocks or flags violations before merge.

Time saved: ~3 hours/week per auditor-facing engineer.

Play 3: Self-Healing Runbooks

Trigger: A known incident pattern is detected (disk full, service crash-loop, cert expiry).

Automation: A runbook automation engine executes remediation steps automatically — restart service, rotate certificate, scale replica count. Human is notified after resolution.

Time saved: ~4 hours/week in reduced incident response time.

Total: 10–12 hours/week recovered. That's a full shift back to strategic work.

~~Strategy 3: Build a Security Culture Without Fear~~

The strongest security posture is worthless if your team is afraid to report issues. Fear-driven security cultures breed cover-ups and burnout. Here's how to build the opposite.

Blameless Post-Mortems

Every incident ends with a post-mortem that asks "what broke in the system?" not "who broke it?" The goal is to find the process gap, not the scapegoat. Publish them internally (anonymized if needed) so the whole org learns.

Security Champions in Every Team

Instead of a centralised security team that reviews everything, embed a security champion in each product team. They get training, tooling access, and a direct line to the ops security lead. No separate function — security is everyone's job.

Metrics That Track Progress, Not Punish Mistakes

Measure mean time to detect (MTTD), mean time to resolve (MTTR), and vulnerability remediation rate. Never track individual error counts. Share dashboards publicly within the org — transparency builds trust.

The "You Found It, You Own It" Policy

Whoever discovers a vulnerability owns the fix — with full support from the team. This turns finding issues into a point of pride, not a burden. Reward finders with recognition, not more work.

~~Strategy 4: The Open Source Toolchain I Actually Use~~

Proprietary security tools lock you into their roadmap and pricing. Open source gives you transparency, community-driven threat intelligence, and full control over your data. Here's the stack I run in production.

Wazuh (SIEM)

Endpoint detection, log analysis, and file integrity monitoring. Deploy the agent on every server. Centralised dashboard for alerts and compliance reports.

VulnClaw (Pentesting)

My own open-source AI-driven penetration testing CLI. Automates recon, exploitation, and reporting. Integrates with CI/CD for pre-merge security gates.

OpenVAS (Scanning)

Vulnerability scanning on a weekly cadence. Schedule scans against your internal and external ranges. Feed results into Grafana for trend tracking.

Grafana (Dashboards)

Unified observability. Pipe in Wazuh alerts, OpenVAS findings, and VulnClaw reports into a single pane of glass. Correlate security events with ops metrics.

Docker Compose for Testing

```
version: '3.8'
services:
  wazuh:
    image: wazuh/wazuh-oss:latest
    ports:
      - "55000:55000"
      - "1514:1514/udp"
  openvas:
    image: greenbone/openvas:latest
    ports:
      - "9392:9392"
  grafana:
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
  vulnclaw:
    build: ./vulnclaw
    depends_on: [wazuh]
```

~~Strategy 5: My Personal Incident Response Checklist~~

When something goes wrong, you don't have time to think. You follow a checklist. This is mine — refined after dozens of real incidents.

- 1. Detect & validate** — Is this a real incident or a false positive? Cross-check with at least two data sources before declaring.
- 2. Classify severity** — P1 (service down / data breach) through P4 (cosmetic issue, no user impact). Severity drives response speed.
- 3. Assemble response team** — Pull in the on-call engineer, subject matter expert, and ops lead. Use a dedicated Slack channel.
- 4. Contain** — Stop the bleeding. Isolate affected systems, roll back bad deployments, block malicious IPs. Speed over perfection.
- 5. Eradicate** — Remove the root cause. Patch the vulnerability, delete the malware, rotate compromised credentials.
- 6. Recover** — Restore service to normal. Verify through monitoring that all systems are healthy.
- 7. Document timeline** — Every action, timestamped. This becomes the backbone of your post-mortem.
- 8. Root cause analysis** — Dig into why it happened. Use the 5 Whys. Find the systemic gap, not the surface error.
- 9. Implement preventive measures** — Deploy the fix, update runbooks, add monitoring alerts so this never happens again.
- 10. Post-mortem** — Blameless, actionable, shared. What went well? What went wrong? What will we change?

About Riccardo Bozzato

Riccardo Bozzato is a Head of Operations and AI Security Builder with a track record of building secure, high-velocity infrastructure. He has spent years bridging the gap between operations and security — proving that the two disciplines are strongest when they work in lockstep.

He is the creator of VulnClaw, an AI-driven penetration testing CLI, and Trova, an intelligent search and discovery tool. Both projects are open source and reflect his belief that security tooling should be accessible, transparent, and community-driven.

When not architecting ops pipelines or pentesting infrastructure, Riccardo writes about security operations, automation, and building resilient systems.

Contact & Links

Email: riccardobozzato@gmail.com

LinkedIn: linkedin.com/in/riccardobozzato

GitHub: github.com/0x1717

VulnClaw: github.com/0x1717/VulnClaw

Trova: github.com/0x1717/Trova
